

CS3212 Algorithms

Aya Zirikly
10/06/2026

Primary Tree Operations

Search(k): Standard BST lookup + splay target node (or last visited node if not found) to root.

Insert(k): Standard BST placement + splay newly inserted node k to root.

Delete(k):

1. Search for k (splays k to root).
2. Drop root k. (Leaves left subtree L and right subtree R).
3. Find maximum element in L and splay it to L's root.
4. Attach R as the right child of L's root.

Time Complexity & Amortized Analysis

Worst-Case (Single Operation): $O(N)$

- *Why?* Linear trees can form if nodes are accessed without proper balance invariants.

Amortized Cost (Sequence of M operations): $O(\log N)$

- *Why?* One expensive $O(N)$ operation restructures and flattens the tree, making future operations drastically cheaper.

Static Optimality / Cache Property: Frequently accessed elements run in **$O(1)$ amortized time**.

What does amortized time mean?

- ❑ Amortized time is the average time spent per operation over a long sequence of operations, guaranteed even in the worst-case scenario for the entire sequence.
- ❑ It prevents a single rare, expensive operation from giving a misleading impression of a data structure's overall performance.

What does amortized time mean?

- ❑ Imagine an admission pass to an amusement park: Rides 1 through 9 are free and instant (1 second each). Ride 10 requires a mandatory 30-second maintenance delay.
- ❑ If you evaluate only Ride 10, the worst-case time is 30 seconds. However, across all 10 rides, you spent a total of 39 seconds (9 + 30). The amortized time per ride is:

$$\text{Amortized Time} = 39 \text{ seconds} / 10 \text{ rides} = 3.9 \text{ seconds per ride}$$

- ❑ The cheap operations "absorb" the cost of the rare expensive one.

Time Complexity

Why Worst-Case is $O(N)$:

- ❑ Unlike AVL, Splay Trees have no strict height invariants. If you insert elements in sorted order (1, 2, 3, 4, 5) without splaying during insertion, the tree becomes a linked list of height N . Searching for the deepest element takes $O(N)$ steps.

Time Complexity

The Magic of Splaying:

- ❑ A single $O(N)$ search is expensive, but the act of splaying that deep node to the root halves the height of the entire search path (thanks to Zig-Zig rotations).
- ❑ The single expensive $O(N)$ operation "pays forward" by making the next several operations drastically faster.
- ❑ Any sequence of M operations on a tree of size N takes $O(M \log N)$ total time.
- ❑ Locality of Reference (Static Optimality):
 - ❑ If a small subset of k keys is accessed 90% of the time, those k keys remain near the root. Accessing them takes $O(1)$ **amortized time**, outperforming even AVL and Red-Black trees for non-uniform access patterns.

Why the Search Takes $O(N)$ Steps - example

If you search for key 5 in that skewed tree:

1. The algorithm starts at the root (1).
2. It traverses down the right pointers: 1 -> 2 -> 3 -> 4 -> 5.
3. To reach key 5, it must traverse through all N nodes in the tree.

Traversing N pointers requires **N steps**, making the traversal cost **$O(N)$** for that specific operation.

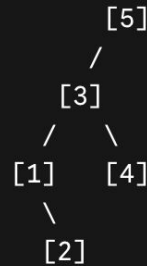
Why the Search Takes $O(N)$ Steps - example

If you search for key 5 in that skewed tree:

1. The algorithm starts at the root (1).
2. It traverses down the right pointers: 1 -> 2 -> 3 -> 4 -> 5.
3. To reach key 5, it must traverse through all N nodes in the tree.

Traversing N pointers requires **N steps**, making the traversal cost **$O(N)$** for that specific operation.

After searching for 5 and splaying it to the root:



Connecting Worst-Case to Amortized

Once key 5 is found at depth N , the algorithm **splays** key 5 to the root using Zig-Zig rotations:

- ❑ The Worst-Case Operation: The search down to 5 took $O(N)$ time.
- ❑ The Amortized Recovery: The splay operation restructured the tree, ~halving the depth of the remaining nodes.

Time Complexity

Operation	Worst-Case (Single Operation)	Amortized Cost (Single Operation)
Search	$O(N)$	$O(\log N)$
Insert	$O(N)$	$O(\log N)$
Delete	$O(N)$	$O(\log N)$

Pros vs. Cons

Pros: Simple node layout (zero metadata space overhead), exceptionally fast for non-uniform access patterns, simple code base.

Cons: Individual operations can be $O(N)$ worst-case, read operations modify the tree (requires write locks in multi-threaded environments).

The 3 Methods of Amortized Analysis

The Aggregate Method

- **Concept:** Calculate the upper-bound total cost $T(N)$ for any sequence of N operations, then divide by N to get the average cost per operation.
- **Formula:** Amortized Cost = $T(N) / N$.
- **Perspective:** A simple "brute-force" global averaging approach without tracking individual operation details.

We insert a sequence of N unsorted elements into an initially empty tree one by one.

- **Cost of 1 insertion:** The depth of the search path (number of node comparisons/rotations).

Worst-Case vs. Amortized Analysis

- **Naive Worst-Case Analysis:**
 - In a degenerate un-balanced tree, a single insertion can take linear time: $O(N)$ work.
 - Naively, N worst-case insertions * $O(N)$ work = **$O(N^2)$ total cost.**
- **Aggregate Analysis (Summing Total Work Across All Insertions):**
 - **Step 1 (Sum total cost):** In a self-balancing / splaying tree, inserting N elements creates a balanced structure where the depth of the k -th insertion is bounded by $\log_2(k)$.
 - Total work $T(N) = \log_2(1) + \log_2(2) + \log_2(3) + \dots + \log_2(N)$
 - $T(N) = \log_2(N!) < N * \log_2(N)$ **total operations.**

We insert a sequence of N unsorted elements into an initially empty tree one by one.

- **Cost of 1 insertion:** The depth of the search path (number of node comparisons/rotations).

Worst-Case vs. Amortized Analysis

- **Naive Worst-Case Analysis:**
 - In a degenerate un-balanced tree, a single insertion can take linear time: $O(N)$ work.
 - Naively, N worst-case insertions * $O(N)$ work = **$O(N^2)$ total cost.**
- **Aggregate Analysis (Summing Total Work Across All Insertions):**
 - **Step 1 (Sum total cost):** In a self-balancing / splaying tree, inserting N elements creates a balanced structure where the depth of the k -th insertion is bounded by $\log_2(k)$.
 - Total work $T(N) = \log_2(1) + \log_2(2) + \log_2(3) + \dots + \log_2(N)$
 - $T(N) = \log_2(N!) < N * \log_2(N)$ **total operations.**
 - **Step 2 (Average):** Amortized Cost = $T(N) / N$
 - Amortized Cost = $(N * \log_2(N)) / N = O(\log N)$ **amortized cost per insertion.**

$$\log_2(k) \leq \log_2(N) \text{ for all } k \leq N$$

The Accounting Method (Banker's Method)

Concept: Assign an artificial "amortized cost" (charge) to each operation type.

Mechanism:

- Overcharge cheap operations and store the extra amount as pre-paid "credits" (tokens) on specific data objects.
- Spend these stored credits later to pay for expensive maintenance operations without taking a deficit.

Example: Dynamic Array Resizing (Accounting Method)

The Problem

- Appending an item when capacity is available takes 1 step (cheap operation).
- Appending when the array is full takes N steps to allocate a new array of size $2N$ and copy N elements (expensive operation).

Why Amortized Cost is $O(1)$

- After an array doubles from size N to $2N$, inserting N new items collects $3N$ total tokens.
- When the array fills up at capacity $2N$, copying all $2N$ items costs exactly $2N$ tokens—paid entirely using the stored credits!
- Because every insert charges a constant 3 tokens, the amortized cost per insert is **$O(1)$** .

The Potential Method (Physicist's Method)

Concept: Define a global function Φ that measures the total "pre-paid energy" or "stored potential" of the entire data structure.

Mechanism: Amortized cost equals actual cost plus the change in potential. Cheap operations increase potential; expensive operations drain potential.

The Potential Method (Physicist's Method)

Mechanism: Amortized cost equals actual cost plus the change in potential. Cheap operations increase potential; expensive operations drain potential.

Splay Tree Potential Function (High Level)

The Rank-Based Potential Function

- Assign a **Size** $s(x)$ to every node x : the total number of nodes in x 's subtree (including x).
- Assign a **Rank** $r(x)$ to every node x : The base-2 logarithm of its size, $r(x) = \log_2(s(x))$.
- **Global Potential $\Phi(T)$** : The sum of ranks $r(x)$ across all nodes x in the entire splay tree T .

Splay Tree Potential Function (High Level)

```
[x]          <-- Height = 6, Size = 7, Rank =  $\log_2(7) \approx 2.81$ 
/
[A]
/
[B]
...
```

```
    [x]          <-- Height = 2, Size = 7, Rank =  $\log_2(7) \approx 2.81$ 
   /  \
  [A]  [B]
 /  \  /  \
[1] [2][3] [4]
```

The Potential Method (Phi)

Key Definitions

- Let c_i be the **actual cost** of the i -th operation.
- Let $\Phi(D_i)$ be the **potential function** representing stored energy in data structure state D_i .
- **Amortized Cost Equation:** $a_i = c_i + \Delta \Phi_i$
 - $\Delta \Phi_i = \Phi(D_i) - \Phi(D_{i-1})$ (the change in potential after operation i).

The Bounding Property

- Total Amortized Cost = Total Actual Cost + $\Phi(D_n) - \Phi(D_0)$.
- As long as the potential function satisfies **$\Phi(D_i) \geq \Phi(D_0)$** for all i (potential never drops below the starting level)
 - The sum of amortized costs is **always greater than or equal to** the sum of actual costs.
- **Conclusion:** Amortized cost acts as a valid, mathematically sound upper bound for actual operational work.

The Potential Method (Phi)

Key Definitions

- Let c_i be the **actual cost** of the i -th operation.
- Let $\Phi(D_i)$ be the **potential function** representing stored energy in data structure state D_i .
- **Amortized Cost Equation:**

$$a_i = c_i + \Delta\Phi_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

$\Delta\Phi_i$: the change in potential after operation i).

The Bounding Property

Summing Over n Operations

Summing a_i for $i = 1$ to n :

$$\sum_{i=1}^n a_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$$

$$\sum_{i=1}^n a_i = \sum_{i=1}^n c_i + \sum_{i=1}^n (\Phi(D_i) - \Phi(D_{i-1}))$$

Splitting the summation:

The Bounding Property

Summing Over n Operations

Summing a_i for $i = 1$ to n :

$$\sum_{i=1}^n a_i = \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1}))$$

Splitting the summation:

$$\sum_{i=1}^n a_i = \sum_{i=1}^n c_i + \sum_{i=1}^n (\Phi(D_i) - \Phi(D_{i-1}))$$

Expanding the telescoping potential terms:

$$(\Phi(D_1) - \Phi(D_0)) + (\Phi(D_2) - \Phi(D_1)) + \cdots + (\Phi(D_n) - \Phi(D_{n-1}))$$

All intermediate terms cancel out, leaving:

$$\sum_{i=1}^n a_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$$

The Bounding Property

Upper Bound Condition

Rearranging for total actual cost:

$$\sum_{i=1}^n c_i = \sum_{i=1}^n a_i - (\Phi(D_n) - \Phi(D_0))$$

If $\Phi(D_n) \geq \Phi(D_0)$ (or more generally $\Phi(D_i) \geq \Phi(D_0)$ for all i) $\Rightarrow \mathbf{\Phi(D_n) - \Phi(D_0) \geq 0}$

$$\sum_{i=1}^n a_i \geq \sum_{i=1}^n c_i$$

The Potential Method (Phi)

Key Definitions

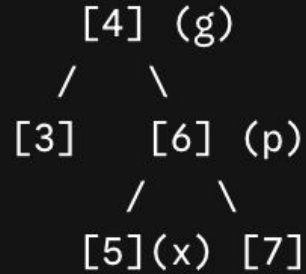
- Let c_i be the **actual cost** of the i -th operation.
- Let $\Phi(D_i)$ be the **potential function** representing stored energy in data structure state D_i .
- **Amortized Cost Equation:** $a_i = c_i + \Delta \Phi_i$
 - $\Delta \Phi_i = \Phi(D_i) - \Phi(D_{i-1})$ (the change in potential after operation i).

The Bounding Property

- Total Amortized Cost = Total Actual Cost + $\Phi(D_n) - \Phi(D_0)$.
- As long as the potential function satisfies **$\Phi(D_i) \geq \Phi(D_0)$** for all i (potential never drops below the starting level)
 - The sum of amortized costs is **always greater than or equal to** the sum of actual costs.
- **Conclusion:** Amortized cost acts as a valid, mathematically sound upper bound for actual operational work.

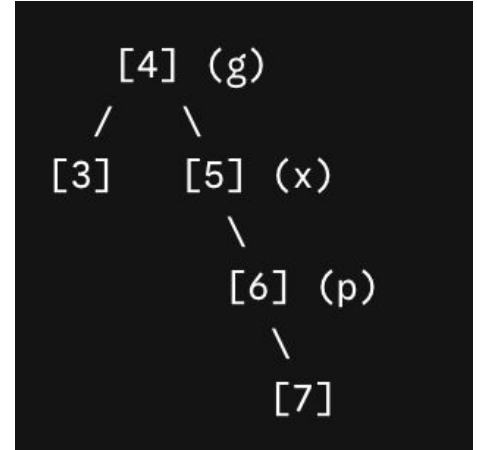
Initial State (Splay Node 5/x)

- ❑ **Target node 5** is the left child of parent 6, and parent 6 is the right child of grandparent 4 (a Zig-Zag or Zag-Zig configuration).
- ❑ **Height:** Distance down to the deepest leaf (edges).
- ❑ **Subtree Size $s(v)$:** Total number of nodes in that node's subtree.
- ❑ **Rank $r(v)$:** Calculated as $\log_2(s(v))$.



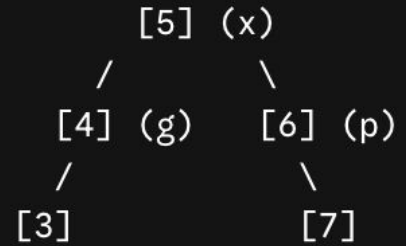
First Rotation: Right-Rotate 5 around Parent 6

- ❑ Node 5 moves into parent 6's position under grandparent 4.
- ❑ Node 6 becomes node 5's right child, taking node 7 along with it as its right child.



Second Rotation: Left-Rotate 5 around Grandparent 4

- ❑ Node 5 moves to the top root position.
- ❑ Former grandparent 4 becomes node 5's left child (carrying node 3 with it).
- ❑ Former parent 6 remains node 5's right child (carrying node 7 with it).



Node	Height Before	Height After	Size $s(v)$ Before	Size $s(v)$ After	Rank $r(v)$ Before	Rank $r(v)$ After	Rank Change (Delta r)
Node 5 (x)	1	2	1	5	0	2.32	+2.32 (Increased)
Node 4 (g)	3	1	5	2	2.32	1	-1.32 (Decreased)
Node 6 (p)	2	1	3	2	1.58	1	-0.58 (Decreased)
Node 3	0	0	1	1	0	0	0.00 (Unchanged)
Node 7	0	0	1	1	0	0	0.00 (Unchanged)

Calculate Total Potential Change (Delta_Phi_i)

Global potential Phi is the sum of ranks of all nodes in the tree. Since only nodes 5, 4, and 6 changed their ranks, we calculate the net change in potential directly:

```
Delta_Phi_i = Sum of Ranks After - Sum of Ranks Before
```

```
Delta_Phi_i = (Rank_After(5) - Rank_Before(5))  
              + (Rank_After(4) - Rank_Before(4))  
              + (Rank_After(6) - Rank_Before(6))
```

```
Delta_Phi_i = (2.32 - 0.00) + (1.00 - 2.32) + (1.00 - 1.58)
```

```
Delta_Phi_i = (+2.32) + (-1.32) + (-0.58)
```

```
Delta_Phi_i = +0.42
```

Calculate Amortized Cost (a_i)

Using the fundamental potential equation

Amortized Cost (a_i) = Actual Cost (c_i) + Potential Change ($\Delta\Phi_i$)

$$a_i = 2 + 0.42$$

$$a_i = 2.42$$

Prove the Boundary Holds (Sum of Amortized Costs $\geq$ Sum of Actual Costs)

To verify that the amortized estimate correctly bounds the real physical work:

$$\text{Amortized Cost} - \text{Actual Cost} = \Delta_{\Phi_i} \quad 2.42 - 2.00 = +0.42 \geq 0$$

Over any sequence of operations from initial state D_0 to final state D_n :

$$\text{Total Amortized Cost} = \text{Total Actual Cost} + \Phi(D_n) - \Phi(D_0)$$

Because all individual subtree sizes are at least 1, every rank $r(v) = \log_2(s(v))$ is greater than or equal to 0.

$$\Phi(D_n) \geq \Phi(D_0)$$

$$\Phi(D_n) \geq 0$$

$$\text{Total Amortized Cost} \geq \text{Total Actual Cost}$$

The amortized cost of 2.42 fully covers the 2 physical rotations while leaving a positive credit balance in the system.

Module 3: Key Lookup, Hashing & Tries

What is Key Lookup?

Definition: Searching for a specific record (Value) using an identifier (Key).

- Example: Looking up a student record by Student ID, or a user profile by Username.

The Dictionary / Map Abstraction:

- Fundamental operations: `Insert(key, value)`, `Lookup(key)`, `Delete(key)`.

Why Traditional Structures Struggle at Scale:

- Unsorted Array / List: Search takes **$O(N)$** time (scans every element).
- Sorted Array / Binary Search Tree: Search takes **$O(\log N)$** time (divides search space).
- **The Goal of Hashing:** Achieve **$O(1)$ constant-time** immediate lookup regardless of how many items exist.

What is Hashing?

The Core Idea: Transforming arbitrary data (strings, objects, large IDs) into a precise array index number using a mathematical function.

How it Works:

1. Take a key (e.g., "alice@example.com").
2. Pass it into a **Hash Function** $h(\text{key})$.
3. The function computes an integer slot index (e.g., slot 412).
4. Jump directly to `Array[412]` in $O(1)$ time.

Analogy: Instead of flipping through a 1,000-page textbook page by page, a magic index tells you instantly: "Go directly to page 412 for your answer."

Real-World Applications of Hashing

❑ Everyday Programming Languages:

- ❑ Python `dict` and `set`
- ❑ Java `HashMap` and `HashSet`
- ❑ C++ `std::unordered_map`
- ❑ JavaScript `Object` and `Map`

❑ High-Performance Caching & Databases:

- ❑ **Memcached & Redis:** In-memory key-value stores holding web session data.
- ❑ **Database Indexing:** Fast row retrieval in SQL / NoSQL databases.

❑ Systems Infrastructure:

- ❑ **Compilers:** Symbol tables looking up variable names and functions.
- ❑ **Load Balancing:** Consistent hashing to distribute network traffic across servers.

- ❑ *(Note: Algorithmic key-lookup hashing is optimized for fast index generation, unlike cryptographic hash functions like SHA-256 which are designed for security).*

Direct Addressing vs. Hashing Framework

Direct Addressing (Key IS the Index):

- Store item k at $\text{Array}[k]$. Perfect $O(1)$ worst-case speed.
- **Limitation:** Only works if the set of possible keys is tiny (e.g., ASCII characters 0–255).

The Universe Problem:

- If keys are 9-digit Social Security Numbers, Direct Addressing requires an array of 1,000,000,000 slots even if you only store 100 people.

The Hashing Solution:

- Map a huge key space (Universe U) into a small, realistic table size M using $h(\text{key}) = \text{key} \bmod M$.
- **Trade-off:** Since U is vastly larger than M , two different keys will eventually map to the same slot (**Collision**).

Handling Data Collisions

- ❑ **Pigeonhole Principle:** If 10 items are placed into 7 slots, at least one slot must hold more than one item.
- ❑ **Strategy 1: Separate Chaining**
 - ❑ Each table slot holds a pointer to a Linked List of items that hashed to that same index.
 - ❑ *Pros:* Simple, table never "fills up".
- ❑ **Strategy 2: Open Addressing**
 - ❑ All elements live directly inside the array. If slot $h(k)$ is occupied, search for another open slot.
 - ❑ **Linear Probing:** Check slot $h(k) + 1$, $h(k) + 2$, etc.
 - ❑ **Double Hashing:** Use a second hash function to calculate the jump step size.

Open Addressing Example (Linear Probing Example)

Setup:

- ❑ **Table Size (M):** 7 slots (indices 0 to 6)
- ❑ **Hash Function:** $h(\text{key}) = \text{key} \bmod 7$
- ❑ **Probing Rule:** If slot $h(\text{key})$ is occupied, probe $(h(\text{key}) + i) \bmod 7$ for $i = 1, 2, 3 \dots$

Open Addressing Example (Linear Probing Example)

Step 1: Insert 12

- ❑ Hash: $12 \bmod 7 = 5$. Slot 5 is empty. Place 12 at index 5.
- ❑ **Table:** [_ | _ | _ | _ | _ | 12 | _]

Step 2: Insert 19

- ❑ Hash: $19 \bmod 7 = 5$. **Collision at slot 5!**
- ❑ Probe $i = 1$: $(5 + 1) \bmod 7 = 6$. Slot 6 is empty. Place 19 at index 6.
- ❑ **Table:** [_ | _ | _ | _ | _ | 12 | 19]

Open Addressing Example (Linear Probing Example)

Step 3: Insert 26

- ❑ Hash: $26 \bmod 7 = 5$. **Collision at slot 5!**
- ❑ Probe $i = 1$: $(5 + 1) \bmod 7 = 6$. **Collision at slot 6!**
- ❑ Probe $i = 2$: $(5 + 2) \bmod 7 = 0$ (*wraps around*). Slot 0 is empty. Place 26 at index 0.
- ❑ **Table:** [26 | _ | _ | _ | _ | 12 | 19]

Step 4: Looking up Key 26 (Search Process)

- ❑ Calculate initial hash: $26 \bmod 7 = 5$.
- ❑ Inspect slot 5: Contains 12 (*not a match, keep probing*).
- ❑ Inspect slot 6: Contains 19 (*not a match, keep probing*).
- ❑ Inspect slot 0: Contains 26 (*Match found! Return success*).

Key takeaways

Probing Path: Lookups must trace the exact same collision search path as insertions.

Primary Clustering: Nearby occupied slots form contiguous blocks (slots 5, 6, 0), making subsequent insertions take longer.

Termination Condition: A search stops either when the key is found or when an **empty slot** is hit (meaning the key isn't in the table).

Table Load Factor & Performance

The Intuition (Occupancy Rate):

- ❑ Load factor measures **how full or crowded** the hash table currently is.
- ❑ Think of it like parking lot capacity: as the lot fills up, finding an open space takes longer.

The Load Factor Formula:

- ❑ $\alpha = N / M$
- ❑ N = Total number of elements stored in the table
- ❑ M = Total number of available array slots
- ❑ *Example:* 75 items stored in 100 slots → $\alpha = 0.75$ (75% capacity)

Table Load Factor & Performance

Performance Thresholds:

- ❑ **Separate Chaining:** Works fine even if $\alpha > 1.0$ (lists just grow longer), but average search time scales to $O(1 + \alpha)$.
- ❑ **Open Addressing:** Performance drops drastically as α approaches 1.0 because empty slots become rare.

Dynamic Resizing Rule:

- ❑ When α crosses a threshold (typically 0.75), allocate a larger array (usually double the size $2M$) and **rehash** all keys to reset α back to ~ 0.375 .

Why Open Addressing Slows Down Near $\alpha = 1.0$ Extra

The Expected Probes Formula:

- Under uniform hashing, the expected number of probes needed to find an open slot is: Expected Probes
approx $1 / (1 - \alpha)$

Non-Linear Probe Explosion:

Extra

Load Factor (alpha)	Array Fullness	Expected Probes to Find Empty Slot
0.5	50% full	$1 / (1 - 0.50) = 2$ probes
0.75	75% full	$1 / (1 - 0.75) = 4$ probes
0.9	90% full	$1 / (1 - 0.90) = 10$ probes
0.95	95% full	$1 / (1 - 0.95) = 20$ probes
0.99	99% full	$1 / (1 - 0.99) = 100$ probes

Primary Clustering & Missing Key Overhead

Primary Clustering ("The Rich Get Richer"):

- With linear probing, contiguous blocks of filled slots form long clusters.
- A cluster of length C has a higher probability $(C + 1) / M$ of absorbing the next hash hit, causing clusters to merge into mega-blocks.

Search Overhead for Missing Keys:

- In open addressing, searching for a key that **does not exist** only stops when an **empty slot** is found.
- At $\alpha = 0.98$, missing key lookups degrade from $O(1)$ constant time to near $O(N)$ linear array scans.